

1

Linear Regression

Pre-processing

- **Normalize** each feature: $x^{(d)} \leftarrow \frac{x^{(d)} - \mu_d}{\sigma_d}$
- **Add bias**: append 1 to each sample, so $D \leftarrow D + 1$

MSE Risk

$$R(w) = \frac{1}{N} \sum_{i=1}^N (w^T x_i - y_i)^2$$

Gradient

$$\nabla_w R(w) = \frac{2}{N} \sum_{i=1}^N (w^T x_i - y_i) x_i$$

Closed-Form Solution

$$w^* = (X^T X)^{-1} X^T y$$

Multi-output: $W^* = X^\dagger Y$

Test

$$\hat{y} = w^{*T} x \text{ (} C=1 \text{)} \quad \hat{y} = W^{*T} x \text{ (} C>1 \text{)}$$

2

Regression Evaluation

- **MSE**: $\frac{1}{N} \sum (y_i - \hat{y}_i)^2$ — penalizes large errors
- **RMSE**: $\sqrt{\frac{1}{N} \sum (y_i - \hat{y}_i)^2}$ — same unit as y
- **MAE**: $\frac{1}{N} \sum |\hat{y}_i - y_i|$ — robust to outliers
- **MAPE**: $\frac{1}{N} \sum \left| \frac{\hat{y}_i - y_i}{y_i} \right|$ — relative; undef. if $y_i = 0$

3

Logistic Regression (Binary)

Pre-processing

Same as linear regression: normalize + add bias.

Sigmoid

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \quad \hat{y}_i = \sigma(w^T x_i) \in (0, 1)$$

Cross-Entropy Loss

$$R(w) = - \sum_{i=1}^N \left[y_i \ln \hat{y}_i + (1 - y_i) \ln (1 - \hat{y}_i) \right]$$

Gradient

$$\nabla_w R(w) = \sum_{i=1}^N (\hat{y}_i - y_i) x_i$$

No closed form $\Rightarrow$ use **gradient descent**.

Prediction

$$\hat{y} = \begin{cases} 1 & \text{if } \sigma(w^{*T} x) \geq 0.5 \\ 0 & \text{otherwise} \end{cases}$$

4

Multi-Class Classification

One-Hot Encoding

$y_i \in \{0, 1\}^C$: one entry is 1, rest are 0.

Softmax (Logistic)

$$\hat{y}^{(k)}(x) = \frac{\exp(w^{(k)T} x)}{\sum_{j=1}^C \exp(w^{(j)T} x)}$$

Multi-Class Cross-Entropy

$$R(W) = - \sum_{i=1}^N \sum_{k=1}^C y_i^{(k)} \ln \hat{y}_i^{(k)}$$

Gradient

$$\nabla_W R(W) = \sum_{i=1}^N x_i (\hat{y}_i - y_i)^T$$

Prediction: $k = \arg \max_j \hat{y}^{(j)}$

5

Classification Evaluation

	True Pos	True Neg
Pred Pos	TP	FP
Pred Neg	FN	TN

- **Accuracy**: $\frac{TP+TN}{TP+TN+FP+FN}$ — misleading if im-balanced
- **Precision**: $\frac{TP}{TP+FP}$ — prioritize if false alarms costly
- **Recall**: $\frac{TP}{TP+FN}$ — prioritize if misses costly
- **FPR**: $\frac{FP}{FP+TN}$
- **F1**: $2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$ — use when imbalanced

Multi-class: $C \times C$ matrix;

6

k-Nearest Neighbors

- **Euclidean**: $\sqrt{\sum_k (x_i^{(k)} - x_j^{(k)})^2}$
- **Chi-square**: $\sqrt{\sum_k \frac{(x_i^{(k)} - x_j^{(k)})^2}{x_i^{(k)} + x_j^{(k)}}$

Classification

Find k nearest neighbors $\Rightarrow$ majority vote.

Regression

Find k nearest $\Rightarrow$ (weighted) average:

$$\hat{y} = \frac{\sum_i w_i y_i}{\sum_i w_i}, \quad w_i = \frac{1}{d_i}$$

7

Feature Expansion & Regularization

Polynomial Feature Expansion

Expansion

$\phi: \mathbb{R}^D \rightarrow \mathbb{R}^F$ maps x to higher-dim features.
 $\hat{y}_i = w^T \phi(x_i)$

Closed-Form with Expansion

$$w^* = (\Phi^T \Phi)^{-1} \Phi^T y$$

Cover's Theorem

A complex classification problem is more likely linearly separable in high-dimensional space (if not densely populated).

Overfitting & Underfitting

- **Overfitting**: low train error, high test error
- **Underfitting**: high error on both

Cross-Validation

- **Val. set**: 80/20 split — cheap, wastes data
- **LOOCV**: train on $N-1$, test on 1, repeat — exact, expensive
- **K-fold**: split into k parts, rotate — best compromise

L2 Regularization (Ridge)

Regularized Objective

$$E(w) = R(w) + \lambda w^T w$$

$\lambda > 0$ controls regularization strength.

Ridge Regression Solution

$$w^* = (\Phi^T \Phi + \lambda I_F)^{-1} \Phi^T y$$

Multi-output: $W^* = (\Phi^T \Phi + \lambda I_F)^{-1} \Phi^T Y$

8

Kernel Methods

Kernel Function

Kernel

$$k(x_i, x_j) = \phi(x_i)^T \phi(x_j)$$

To be valid: $k(x, x) = \|\phi(x)\|^2 \geq 0$ for all x

- **Polynomial**: $(x_i^T x_j + c)^d$
- **Gaussian**: $\exp\left(-\frac{\|x_i - x_j\|^2}{2\sigma^2}\right)$

Kernel Matrix

$$K_{ij} = k(x_i, x_j) \in \mathbb{R}^{N \times N}$$

Kernel Ridge Regression

Kernel Ridge Solution

$$\alpha^* = (K + \lambda I_N)^{-1} y$$

Prediction

$$\hat{y} = y^T (K + \lambda I)^{-1} k(X, x)$$

where $k(X, x) = \left(k(x_1, x), \dots, k(x_N, x)\right)^T$

Multi-output:

$$W^* = \Phi^T (K + \lambda I)^{-1} Y, \quad \hat{y} = Y^T (K + \lambda I)^{-1} k(X, x)$$

Kernel selection

Choose kernel by **cross-validation**. No general best choice.